Practical No. 5

Unix Process Management using Shell Script (ps, fork, exec, wait)

1. AIM

To implement Unix/Linux process management commands and system calls such as ps, fork, wait/join and the exec family using a Shell script.

2. REQUIREMENTS

- Operating System: Ubuntu / Linux / Unix
- Shell: Bash
- Text editor: Nano / Vim / Gedit / VS Code
- Terminal
- GCC compiler (only if demonstrating the C system calls fork(), exec(), wait())
- Hardware: computer, keyboard, monitor

3. SHORT THEORY

- Process: a program that is currently running. Each process has a unique PID (Process ID).
- PPID: Parent Process ID. It is the PID of the process that created this process.
- ps command: shows running processes. ps shows processes of the current terminal; ps -ef shows all processes with full details (PID, PPID, terminal, time, command).
- fork(): system call that creates a new process. The new one is the child, the creator is the parent. It returns 0 in the child and the child's PID (positive) in the parent.
- exec family (execl, execlp, execv, execvp): replaces the current process with a new program. The process keeps its PID but the old program is gone.
- wait(): the parent waits until its child finishes. This gives synchronization and prevents zombie processes.
- join(): the thread version of wait(). One thread waits for another to finish. For processes, wait() is used.
- kill: sends a signal to a process, normally to terminate it.

In Bash there is no direct fork(). Running commands in the background with (...) & works like a fork, because it creates a child process.

4. CODE (file name: process_management.sh)

```
#!/bin/bash
echo "===== UNIX PROCESS MANAGEMENT ====="
while true
do
  echo ""
  echo "1. Display Processes (ps)"
  echo "2. Create Child Process (fork-like)"
  echo "3. Execute Command (exec)"
  echo "4. Wait for Process (wait)"
  echo "5. Kill Process (kill)"
  echo "6. Exit"
  read -p "Enter your choice: " choice

  case $choice in
  1)
```

```
    echo ""
    echo "===== RUNNING PROCESSES ====="
    ps
    ;;
  2)
    echo ""
    echo "===== CREATING CHILD PROCESS ====="
    (
      echo "Child Process Started"
      echo "Child PID: $BASHPID"
      sleep 5
      echo "Child Process Finished"
    ) &
    child_pid=$!
    echo "Parent PID: $$"
    echo "Child PID: $child_pid"
    ;;
  3)
    echo ""
    echo "===== EXEC FAMILY ====="
    echo "Executing 'ls -l' command..."
    exec ls -l
    ;;
  4)
    echo ""
    echo "===== WAIT FOR PROCESS ====="
    (
      echo "Child process is running..."
      sleep 5
      echo "Child process completed."
    ) &
    child_pid=$!
    echo "Waiting for child process: $child_pid"
    wait $child_pid
    echo "Child process finished."
    ;;
  5)
    echo ""
    read -p "Enter PID to kill: " pid
    kill $pid
    if [ $? -eq 0 ]
    then
      echo "Process $pid terminated successfully."
    else
      echo "Unable to terminate process $pid."
    fi
    ;;
  6)
    echo "Exiting..."
    exit 0
    ;;
  *)
    echo "Invalid choice!"
    ;;
  esac
done
```

5. HOW TO EXECUTE (step by step)

Command / Key	What it does
`Ctrl + Alt + T`	Open the Ubuntu terminal.
`cd ~`	Go to the home folder.

Command / Key	What it does
`nano process_management.sh`	Create and open the script file.
`(paste code)`	Paste with Ctrl + Shift + V.
`Ctrl+O, Enter, Ctrl+X`	Save and exit nano.
`chmod +x process_management.sh`	Give execute permission.
`./process_management.sh`	Run the script. A menu appears; enter a choice 1 to 6.

Demo order: choose 1 (see processes), then 2 (note parent and child PID), then 1 again (sleep process is visible), then 4 (wait), then 5 (kill a sleep PID), and 3 last, because exec ends the script. Option 6 exits. Do not kill bash PIDs or the terminal may close.

6. LINE-BY-LINE EXPLANATION

Code	Meaning
`#!/bin/bash`	Shebang. Run this script with Bash.
`while true / do`	Endless loop. The menu keeps coming back until the user chooses Exit.
`echo "1. Display ..."`	Lines that print the menu options on the screen.
`read -p "Enter your choice: " choice`	Shows a prompt and stores what the user types in variable choice.
`case $choice in`	Starts a case (like switch). It compares the value of choice with each option below.
`1) ... ps ;;`	Option 1: ps displays the running processes. ;; ends this option.
`( ... ) &`	Option 2: ( ) groups the commands into a subshell and & runs them in the background. This is the 'fork': a child process is created.
`echo "Child PID: $BASHPID"`	$BASHPID gives the PID of the current subshell, i.e. the child.
`sleep 5`	Pauses 5 seconds so the child stays alive long enough to be seen.
`child_pid=$!`	$! holds the PID of the last background process. Saved in child_pid.
`echo "Parent PID: $$"`	$$ is the PID of the main script, i.e. the parent.
`exec ls -l`	Option 3: exec replaces the script with the ls -l command. After it finishes, the script does not return to the menu.
`wait $child_pid`	Option 4: the parent stops here until that child process finishes.
`read -p ... pid`	Option 5: asks for the PID to be killed.
`kill $pid`	Sends the terminate signal to that PID.
`if [ $? -eq 0 ]`	$? is the exit status of the last command. 0 means success (-eq = equal).
`then / else / fi`	then runs on success (prints terminated). else runs on failure (prints unable). fi closes the if.
`6) exit 0`	Option 6: ends the script. 0 means normal exit.
`*)`	Default case. Runs for any other input and prints Invalid choice.
`esac`	Ends the case block (case spelled backwards).
`done`	Ends the while loop.

7. MEANING OF SYMBOLS

Symbol	Use
`$$`	PID of the current shell / main script (the parent).
`$!`	PID of the last background process started.
`$?`	Exit status of the last command. 0 = success, non-zero = failure.
`$BASHPID`	PID of the current subshell (shows the child's own PID inside ( )).
`&`	Run the command in the background. Works like fork.
`( )`	Runs the commands inside in a separate subshell (a child process).
`;;`	Ends one option in a case statement.
`*)`	Default option in case, matches anything else.
`[ ]`	Test brackets. Keep a space after [and before].
`-eq`	Equal to (comparison for numbers).
`$var`	Value stored in a variable.

8. CONCLUSION

Thus, we have successfully implemented a menu-driven Shell script for Unix process management. We learned to display processes using ps, create a child process using background execution (fork-like), replace a process using exec, synchronize using wait, and terminate a process using kill. We also observed PIDs and parent-child relationships.

9. QUICK VIVA ANSWERS

- Process: a program in execution.
- PID vs PPID: PID is the process's own ID; PPID is its parent's ID.
- Purpose of ps: to show running processes.
- ps vs ps -ef: ps shows only current terminal processes; ps -ef shows all processes in full format.
- fork(): creates a child process.
- fork() returns 0: you are in the child process.
- fork() returns positive value: you are in the parent; the value is the child's PID.
- exec family: replaces the current process with a different program.
- fork vs exec: fork creates a new process (copy); exec replaces the program inside the same process.
- Why wait(): so the parent waits for the child to finish and no zombie process is left.